

REVERSE ENGINEERING



November 20, 2025

“The one where we dig deep into the rabbit hole.”

Credits

Content: Maria Rigaki, Veronica Valeros, Sebastian Garcia
Martin Řepa, Lukáš Forst, Ondřej Lukáš, Muris Sladić

Illustrations: Fermin Valeros

Introduction theme video: Art Fermin Valeros, Production: Veronica Valeros

Design: Veronica Garcia, Veronica Valeros, Ondřej Lukáš

Music: Sebastian Garcia, Veronica Valeros, Ondřej Lukáš

CTU Video Recording: Jan Sláma, Václav Svoboda, Marcela Charvatová

Audio files, 3D prints, and Stickers: Veronica Valeros

CTU Video Recording: Jan Sláma, Václav Svoboda, Marcela Charvatová

Audio files, 3D prints, and Stickers: Veronica Valeros

CLASS DOCUMENT	https://bit.ly/BSY2025-9
WEBSITE	https://cybersecurity.bsy.fel.cvut.cz/
CLASS MATRIX	https://matrix.bsy.fel.cvut.cz/
CLASS CTFD (CTU STUDENTS)	https://ctfd.bsy.fel.cvut.cz/
CLASS PASSCODE FORM (ONLINE STUDENTS)	https://bit.ly/BSY-MOOCPasscode
FEEDBACK	https://bit.ly/BSY-Feedback
LIVESTREAM	https://bit.ly/BSY-Livestream
INTRO Animation	https://bit.ly/BSY-IntroTheme Or you can give us a like on YouTube 😊 Official Intro Theme
VIDEO RECORDINGS PLAYLIST	https://bit.ly/BSY-Recordings

Results from the survey of the last class (14:32)

How was the pace of the class?

20 responses



Parish notices

1. No pioneer prizes this week. 😞
2. MOOC Students, please **start the class 9 environment** in SCL, as it may take up to 5 minutes!
3. If you have not done it already, please download and install the latest [IDA Free](#)¹ (9.2) and [Wireshark](#)² on your computer

Class outline

- [What is Reverse Engineering?](#)
- [Preparations](#)
- [Reversing a Network Protocol](#)
- [Defusing a binary bomb](#) 💣

What is Reverse Engineering?

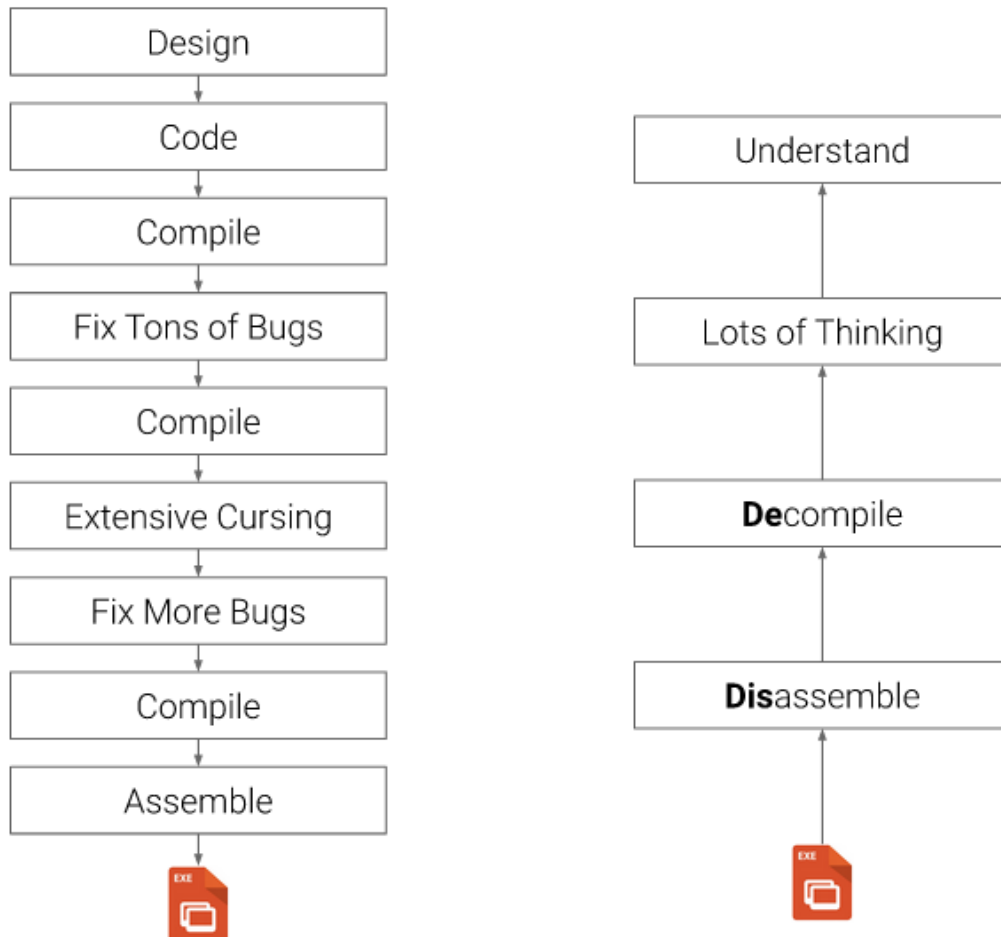
Discover various aspects of reverse engineering and its applications in security.

¹ <https://hex-rays.com/ida-free>

² <https://www.wireshark.org/download.html>

LESSON 9 / REVERSE ENGINEERING

- A **process** of analyzing a (black-box) system to **understand** its design, function, and operation.
- A **critical tool** for researchers, analysts, and security professionals to protect against cyber threats and to improve the security of systems.



"Forward engineering process"³

"Reverse engineering process"

Why do we need it?

There are many cases where we need reverse engineering in **security**, in particular:

- To understand how **malicious** software operates
 - What are its goals and capabilities?
 - How does it spread?

³ Images from <https://pwn.college/cse365-f2023/reverse-engineering>

- What are its vulnerabilities or weaknesses?
- To find **fingerprints** and identify/track threat actors
- To develop countermeasures and defenses against malware

Beyond security:

- Legacy software where the source code is lost
- Understanding proprietary protocols
- Intellectual curiosity and learning!

Breaking "DRM" in Polish trains⁴⁵ 🚅

In 2022, a team of ethical hackers from the Dragon Sector group in Poland was contracted by a Polish rail operator whose trains were mysteriously breaking down after being serviced by independent workshops.



⁴ 📺 37C3 - Breaking "DRM" in Polish trains

⁵ 📺 38C3 - We've not been trained for this: life after the Newag DRM disclosure

The Problem: Several trains manufactured by **Newag** (a Polish rail vehicle company) would not start after undergoing maintenance at third-party repair shops. Newag initially claimed this was due to improper servicing.

The Discovery (Reverse Engineering): The hackers reverse-engineered the train's firmware and found deliberate "lockup mechanisms" or "logic bombs" designed to simulate failures and immobilize the trains.

The Sabotage Mechanisms: The code was designed to trigger malfunctions (such as killing the power or breaking the compressor) if certain conditions were met, including geofencing and date checks.

The Fix: The researchers discovered undocumented, secret key combinations in the driver's cabin that could be used to unlock the trains, a practice for which Newag was allegedly charging the operators tens of thousands of euros.

The aftermath: lawsuits, parliamentary investigations, and so on.

Types of analysis: static vs dynamic

- **Static** analysis: analyze a binary file **without** executing it (at rest).
 - Binary dependent (type of compilation, programming language, architecture)
 - Analyzing assembly language, decompiled code, statically allocated variables, imported libraries, etc.
 - Almost impossible with packed malware
 - Safe for the analyst
- **Dynamic** analysis: analyze a program at **runtime**
 - Using **tracing** tools to trace system and library calls (*strace*, *ltrace*)
 - **Debugging** (*GDB*, *WinDBG*, etc.)
 - **Sandboxes** to simulate a real environment
 - Malware can detect emulation (anti-debug techniques)
 - Might be dangerous

Toolbox

A variety of tools are used during the reversing process, including disassemblers, debuggers, decompilers, sandboxes, system forensics tools, and network analysis tools.

Each platform has a different set of tools:

- **Disassemblers and decompilers:** [Ghidra](#), [IDA](#), [radare2](#) and [cutter](#) (GUI), [binary ninja](#), [Hopper](#) (for Mac), JADX (for Android)
- Useful Linux tools: gdb, objdump, readelf, file, strings, nm, strace, ltrace, and more.
- Windows tools VM: [Flare VM](#) (Mandiant) has everything you need.

In this class, we will be using IDA Free and GDB. IDA is one of the most popular and widely used tools for reversing. It is a disassembler, has a decompiler, a debugger, and many other functionalities. Due to the scope of the class, we are unable to cover some other tools.



Example 1: Reversing a Network Protocol (14:45)

Goal: Learn to reverse engineer a custom command and control protocol.

Host computer. Not in container. ▾

Download the [RAT04.pcap](#) file **on your laptop** from [here](#) (be careful 155MB)

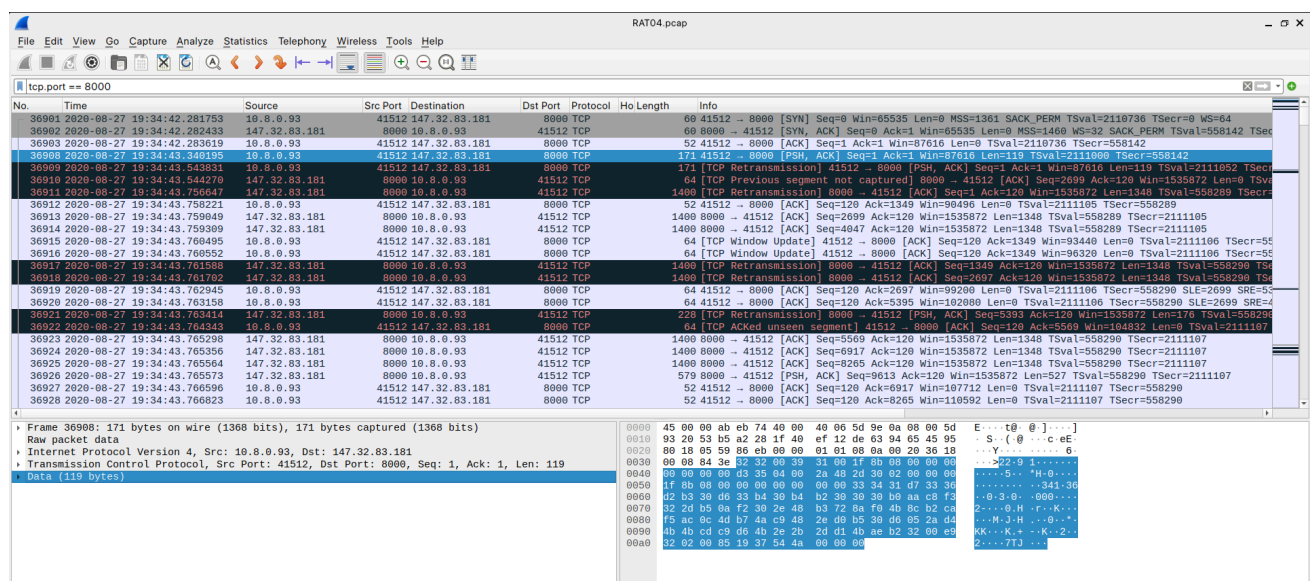
LESSON 9 / REVERSE ENGINEERING

This is a packet capture between a RAT malware (SpyMax) running on an Android phone and a command and control server⁶.

*RAT stands for **Remote Access Trojan**. It is a type of malware that controls a system through a remote network. It is typically installed without the victim's knowledge.*

Open the file with Wireshark. We will focus on the traffic between the mobile phone and the server. Use the following filter and look at the first packet that contains data:

- `tcp.port == 8000`



💡 Take a few minutes and try to see what you can find out! 💡

The first packet that contains data is the packet with the PSH bit (Push flag) on.

The data starts at the byte with the value “0x32” and has a size of 119 bytes.

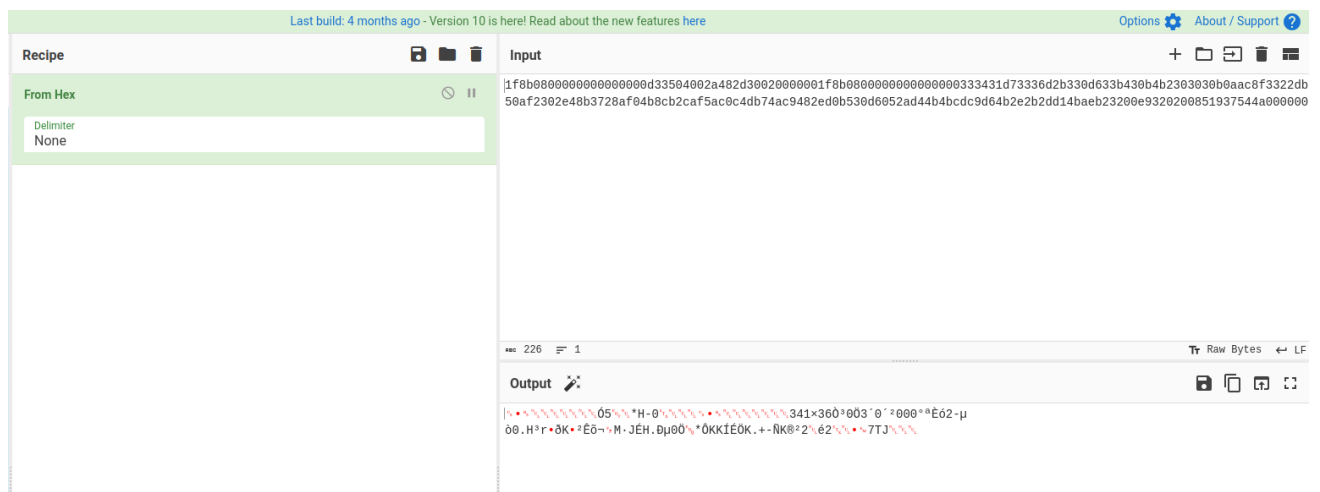
- Are there any interesting patterns in the data?
- What could the bytes at the beginning be? Look at the ASCII representation on the right side.
- Hint: $22 + 91 = 113$, but the total number of bytes in the data are 119!
- What is “**1f 8b**”?

⁶ Dissecting a RAT. Analysis of the SpyMAX.

45 00 00 ab eb 74 40 00 40 06 5d 9e 0a 08 00 5d	E t@ . @ .]]
93 20 53 b5 a2 28 1f 40 ef 12 de 63 94 65 45 95	. S . . (. @ . . . c . eE .
80 18 05 59 86 eb 00 00 01 01 08 0a 00 20 36 18	. . . Y 6 .
00 08 84 3e 32 32 00 39 31 00 1f 8b 08 00 00 00	. . . >22 .9 1
00 00 00 00 d3 35 04 00 2a 48 2d 30 02 00 00 00	 5 . . *H-0
1f 8b 08 00 00 00 00 00 00 00 33 34 31 d7 33 36	 341 .36
d2 b3 30 d6 33 b4 30 b4 b2 30 30 30 b0 aa c8 f3	. . 0 .3 .0 . .000
32 2d b5 0a f2 30 2e 48 b3 72 8a f0 4b 8c b2 ca	2 - . . . 0 .H . r . . K . . .
f5 ac 0c 4d b7 4a c9 48 2e d0 b5 30 d6 05 2a d4	. . . M . J . H . . . 0 . . *
4b 4b cd c9 d6 4b 2e 2b 2d d1 4b ae b2 32 00 e9	KK . . . K . + . - K . . 2 . .
32 02 00 85 19 37 54 4a 00 00 00	2 7TJ

Let's use [CyberChef](https://cyberchef.org/)⁷ to test some ideas:

- Select the data part of the packet.
- Right-click on the part that says *Data (119 bytes)*, select **Copy**, and then “... as a **Hex Stream**”
- Remove the bytes until the “**1f 8b**” and use the “**From Hex**” recipe with delimiter **None**
- What does CyberChef suggest these bytes are?
- Magic bytes: https://en.wikipedia.org/wiki/List_of_file_signatures



Link to a ready-to-use [CyberChef](https://cyberchef.org/) in case you can not do it.

⁷ <https://cyberchef.org/>

- Now that we have a good idea of the data in the first packet, let's check some more packets. Does the pattern continue?
- Use this filter to show only packets that contain data:
 - `(tcp.port == 8000) && (data)`
- Let's go **back** to Wireshark and see if we can find similar packets.

Automating the extraction (16:10)

We notice that the data of a smaller size seems to follow the pattern we noticed in the first packet. We can also filter by size.

- `((tcp.port == 8000) && (data))) && (data.len <= 300)`

Since we have (potentially) hundreds of thousands of packets that follow a similar format, it is best to automate the process. We will do this in our containers.

In the Docker container, we have to go to the example folder:

MOOC Students: SCL HackerLab ▾

Log in to the reversing lab container. From now we will work in the reversing lab container.

- `ssh root@172.20.0.120`
- `Pass: admin`

CTU Students: Docker container. ▾ MOOC Students: SCL Reversing-Lab ▾

- Copy the files for this lab into your container:
 - `cp -r /data/reversing-class/ ./reversing-class`
 - `-r - copy the directory`
- Go to the directory of the first example
 - `cd ~/reversing-class/first_example`

Download the PCAP file to your container with the following command:

- `curl`
`"https://drive.usercontent.google.com/download?id=18v47zh32W-yzI59ZY4_TH7nd79wPODhm&export=download&authuser=0&confirm=t" -o RAT04.pcap`

We will use [tshark](https://tshark.dev/)⁸ to extract the interesting data with the same filter we created in Wireshark:

- `tshark -r RAT04.pcap -Tfields -Y "(tcp.port == 8000) && (data) && (data.len <= 300)" -e data > small_data.log`
 - `-r`: read a file
 - `-Y`: use a display filter
 - `-T fields`: format of text output -> fields
 - `-e data`: field to print -> data

Now that we have all the packets with data smaller than 300 bytes, let's try to automate the decoding process using Python.

For example, we can do it manually for the first packet:

```
root@bsylabs:~/reversing-class/first_example$ python3
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> with open("small_data.log", "r") as f:
...     log_data = f.readlines()
...
>>> log_data[0]
'3232003931001f8b0800000000000000d33504002a482d30020000001f8b0800000000
00000333431d73336d2b330d633b430b4b2303030b0aac8f3322db50af2302e48b3728af
04b8cb2caf5ac0c4db74ac9482ed0b530d6052ad44b4bcd9d64b2e2b2dd14baeb23200e
9320200851937544a000000\n'
>>> int(bytes.fromhex(log_data[0].strip())[:2])
22
>>> byte_stream = bytes.fromhex(log_data[0].strip())
>>> part1 = byte_stream[6:6 + 22]
>>> part2 = byte_stream[6 + 22:]
>>> import gzip
>>> gzip.decompress(part1)
b'-1'
>>> gzip.decompress(part2)
b'147.32.83.181:8000:xnJ5u:RH3pf:BXNaZ:mIyUg:dhcp-83-181.felk.cvut.cz:00
00:2'
```

⁸ <https://tshark.dev/>

Let's check the full script and then run it:

- `cat decoder_small.py`
- `python3 decoder_small.py | less`

💡 Take a few minutes to review the output and see what you can discover! 💡

Let's answer some questions:

1. What kind of information is there?

```
b'147.32.83.181:8000:xnJ5u:RH3pf:BXNaZ:mIyUg:dhcp-83-181.felk.cvut.cz:00
00:2'

b'1x0F0xplugens.angel.plugens.infox0F0xmethodx0F0x1GRU802x0F0xinfox0D0xx
nJ5ux0D0xmIyUg'

b'PING dhcp-83-181.felk.cvut.cz (147.32.83.181) 56(84) bytes of data.---
dhcp-83-181.felk.cvut.cz ping statistics ---1 packets transmitted, 0
received, 100% packet loss, time 0ms'

b'1x0F0xplugens.angel.plugens.filesx0F0xmethodx0F0x5XBL990x0F0xfilesx0D0
xget0'

b'1x0F0xplugens.angel.plugens.filesx0F0xmethodx0F0x-1x0F0xcommendx0D0xcp
-R /storage/emulated/0/DCIM/Camera /storage/emulated/0/DCIM'

b'65.9667x0D0x-18.5333x0D0x0.0x0D0x0'
```

2. Did we manage to reverse all the packets? Do we understand everything?
3. Next steps:
 - a. Separate traffic, add timestamps, and enrich the data extracted from the pcap, etc.
 - b. Look at the bigger packets.
 - c. Look at the failed packets.

- d. Reverse the Android APK file.
 - e. Other?
4. Read more in the [blog](#) "Dissecting a RAT. Analysis of the SpyMAX"⁹¹⁰



Preparations: x64 Assembly and GDB (15:15)

Goal: Learn and/or remind ourselves of registers and calling conventions in x64. Introduce basic usage and concepts of GDB.

⁹ <https://www.stratosphereips.org/blog/2021/2/26/dissecting-a-rat-analysis-of-the-spymax>

¹⁰

https://dspace.cvut.cz/bitstream/handle/10467/94720/F3-BP-2021-Babayeva-Kamila-Kamila_Bachelor_Thesis_RAT_Execution_and_Analysis.pdf?sequence=-1&isAllowed=y

LESSON 9 / REVERSE ENGINEERING

You can find nice x64 cheat sheets [here](#)¹¹, or [here](#)¹².

Let's take a look at the registers we will be encountering later.

	64 bit	32 bit	16 bit	8 bit
A (accumulator)	RAX	EAX	AX	AL
B (base, addressing)	RBX	EBX	BX	BL
C (counter, iterations)	RCX	ECX	CX	CL
D (data)	RDX	EDX	DX	DL
	RDI	EDI	DI	DIL
	RSI	ESI	SI	SIL
Numbered (n=8..15)	Rn	RnD	RnW	RnB
Stack pointer	RSP	ESP	SP	SPL
Frame pointer	RBP	EBP	BP	BPL

The calling convention is:

- Registers **rdi**, **rsi**, **rdx**, **rcx**, **r8**, and **r9** are used to pass the first six integers or pointer parameters to called functions. For floats **xmm0** - **xmm7** registers are used.
- Registers **rbp**, **rbx**, **r12** - **r15** are **nonvolatile** - must be preserved by the callee.
- The rest are **volatile** - a function can overwrite them.
- **rax** - return value register

Examples:

- **mov rax, rdx** - move data from the rdx register to the rax register
- **lea rdi, [rbx+0x10]** - move address rbx+0x10 into the rdi register
- **add rax, rdx** - set rax equal to rax + rdx
- **push rax** - grow stack by 8 bytes, and put contents of the rax on top
- **pop rax** - put the top 8 bytes of the stack in the rax register

¹¹ <https://gist.github.com/justinian/385c70347db8aca7ba93e87db90fc9a6>

¹² <https://learn.microsoft.com/en-us/cpp/build/x64-software-conventions?view=msvc-170>

- **jmp 0x520310** - jump to address 0x520310 and execute the instruction there

Also, install and test GEF¹³ in your containers, which is a set of extensions for the GDB debugger (This is already in your dockers) :

CTU Students: Docker container. ▾

- `wget https://gef.blah.cat/sh`
- `vi sh` (to check it is not malicious code from marik0)
- `bash sh`
- Or, as a one-liner:
 - `bash -c "$(curl -fsSL https://gef.blah.cat/sh)"`

CTU Students: Docker container. ▾

MOOC Students: SCL Reversing-Lab ▾

- Test that GDB works and the (GDB Enhanced Features) **GEF** is installed:
 - `gdb -nx -ex 'pi print(sys.version)' -ex quit`
 - Prints out the Python version that your GDB supports
 - `cat ~/.gdbinit`
- You should see something like that at the bottom of the terminal

```
root@bsylabs:~$ cat ~/.gdbinit
source ~/.gef-2025.01.py
```

¹³ <https://hugsy.github.io/gef/>

Let's take GDB out for a spin.

We will start by creating a simple C code and compiling it.

```
#include <stdio.h>

int return_five(void){
    for(int i = 0; i < 6; i++){
        if(i == 6 - 1) {
            return 5;
        }
    }
}

void main(void){
    int a = 5;

    if(a == return_five())
        printf("Hello World %d\n!", a);
}
```

CTU Students: Docker container. ▾

MOOC Students: SCL Reversing-Lab ▾

- `cd ~/reversing-class/hello_world`
- Let's use gcc to compile the code
 - `gcc -o hello_world hello_world.c`
 - `-o` to set the name of the executable binary
- We can try to run our code:
 - `./hello_world`
- Good! Now we can open it in GDB
 - `gdb ./hello_world`
- We can put a breakpoint at the main function.
 - `b main`
- Now we run the code

o r

```

$rax : 0x00005555555515f → <main+0000> push rbp
$rbx : 0x00007ffffffeb78 → 0x00007ffffffede2 → "/root/hello"
$rcx : 0x0000555555557dd8 → 0x00005555555550f0 → <__do_global_ctors_aux+0000> endbr64
$rdx : 0x00007ffffffeb88 → 0x00007ffffffedee → "SHELL=/bin/bash"
$rsp : 0x00007ffffffea60 → 0x0000000000000001
$rbp : 0x00007ffffffea60 → 0x0000000000000001
$rsi : 0x00007ffffffeb78 → 0x00007ffffffede2 → "/root/hello"
$rdi : 0x1
$rip : 0x000055555555163 → <main+0004> sub rsp, 0x10
$r8 : 0x0
$r9 : 0x00007fffff7fce680 → <_dl_fini+0000> push rbp
$r10 : 0x00007fffff7fca878 → 0x0000c00120000000e
$r11 : 0x00007fffff7fe0a10 → <_dl_audit_preinit+0000> mov eax, DWORD PTR [rip+0x1c422] # 0x7fffff7fce38 <
$r12 : 0x0
$r13 : 0x00007ffffffeb88 → 0x00007ffffffedee → "SHELL=/bin/bash"
$r14 : 0x0000555555557dd8 → 0x00005555555550f0 → <__do_global_ctors_aux+0000> endbr64
$r15 : 0x00007fffff7ffd020 → 0x00007fffff7ffe2e0 → 0x0000555555554000 → jg 0x555555554047
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow resume virtualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00

0x00007ffffffea60 | +0x0000: 0x0000000000000001 ← $rsp, $rbp
0x00007ffffffea68 | +0x0008: 0x00007ffff7e0224a → <__libc_start_call_main+007a> mov
0x00007ffffffea70 | +0x0010: 0x00007ffffffeb60 → 0x00007ffffffeb68 → 0x0000000000000000 (?)
0x00007ffffffea78 | +0x0018: 0x00005555555515f → <main+0000> push rbp
0x00007ffffffea80 | +0x0020: 0x00000000155554040
0x00007ffffffea88 | +0x0028: 0x00007ffffffeb78 → 0x00007ffffffede2 → "/root/hello"
0x00007ffffffea90 | +0x0030: 0x00007ffffffeb78 → 0x00007ffffffede2 → "/root/hello"
0x00007ffffffea98 | +0x0038: 0xfd4f40da5c1693f9

0x5555555515e <return_five+0025> ret
0x5555555515f <main+0000> push rbp
0x55555555160 <main+0001> mov rbp, rsp
➔ 0x55555555163 <main+0004> sub rsp, 0x10
0x55555555167 <main+0008> mov DWORD PTR [rbp-0x4], 0x5
0x5555555516e <main+000f> call 0x55555555139 <return_five>
0x55555555173 <main+0014> cmp DWORD PTR [rbp-0x4], eax
0x55555555176 <main+0017> jne 0x55555555191 <main+50>
0x55555555178 <main+0019> mov eax, DWORD PTR [rbp-0x4]

[#0] Id 1, Name: "hello", stopped 0x55555555163 in main (), reason: BREAKPOINT
[#0] 0x55555555163 → main()
gef> 

```

Registers

Stack

Code

- We can take a look at the disassembled code

- `disass main`

```
gef> disassemble main
Dump of assembler code for function main:
0x00005555555515f <+0>:    push    rbp
0x000055555555160 <+1>:    mov     rbp, rsp
=> 0x000055555555163 <+4>:    sub     rsp, 0x10
0x000055555555167 <+8>:    mov     DWORD PTR [rbp-0x4], 0x5
0x00005555555516e <+15>:   call    0x55555555139 <return_five>
0x000055555555173 <+20>:   cmp     DWORD PTR [rbp-0x4], eax
0x000055555555176 <+23>:   jne     0x55555555191 <main+50>
0x000055555555178 <+25>:   mov     eax, DWORD PTR [rbp-0x4]
0x00005555555517b <+28>:   mov     esi, eax
0x00005555555517d <+30>:   lea     rax, [rip+0xe80]          # 0x555555556004
0x000055555555184 <+37>:   mov     rdi, rax
0x000055555555187 <+40>:   mov     eax, 0x0
0x00005555555518c <+45>:   call    0x55555555030 <printf@plt>
0x000055555555191 <+50>:   nop
0x000055555555192 <+51>:   leave
0x000055555555193 <+52>:   ret
End of assembler dump.
gef> 
```

- The current instruction where the breakpoint is
 - `sub rsp, 0x10`
 - What does this mean?
 - `mov DWORD PTR [rbp-0x4], 0x5`
 - What is the equivalent C code?
 - `call 0x55555555139 <return_five>`
 - What is the equivalent C code?
 - `cmp DWORD PTR [rbp-0x4], eax`
 - What is the equivalent C code?
- We can execute the next instruction by typing one of these
 - `ni` (does not step inside functions)
 - `nexti`
 - `si` (steps inside functions)
 - `stepi`
- There is a call to a `return_five()` function. We can put a breakpoint at that address
 - `b return_five`

- Or we can put a breakpoint at the address:
 - `b *0x<put_the_address_shown_in_gdb>`
 - To see the breakpoints, we can use
 - `info b`
 - To remove a breakpoint
 - `delete <breakpoint_number>`
- We can further analyze the assembly. To continue execution to the next breakpoint
 - `c`
- Let's return to the main function. We can put a new breakpoint on the instruction after the function call to `return_five()`. So we go back to our terminals and look for
 - `cmp DWORD PTR [rbp-0x4], eax`
 - Put the breakpoint
 - `b *0x<put_the_address_shown_in_gdb>`
 - `c`
- The `jne` jump after the current instruction will not be taken since the value of the `eax` register, and the memory location `rbp-0x4` are the same.
 - But if we want to make that jump for some reason, we can use GDB to modify the disassembly. For example, first, we can check what the value is at address `rbp-0x4`
 - `x/w $rbp-0x4`
 - `w` - this means dword
 - Then we can change the value at the address pointed to by `rbp-0x4`
 - `set *<address_from_x/w_command>=0x6`
 - Notice now that the jump is taken

Recap:

- We reminded ourselves of the registers in x64 architecture and how they are passed to functions.
- We went over some basic commands in assembly.
- We got familiar with some basic use cases in GDB.

~~~~  **First Break!**  ~~~~ (15:30, 10m)

## Example 2: Defusing a binary bomb (15:40)

Goal: Reverse a C-based Linux binary using a disassembler/decompiler and a debugger.

A lab exercise from CMU. From the original web page<sup>14</sup>:

*A "binary bomb" is a program provided to students as an object code file. When run, it prompts the user to type in **6 different strings**. If any of these is incorrect, the bomb "explodes," printing an error message...*

*Students must "defuse" their own bomb by disassembling and reverse-engineering the program to determine what the six strings should be.*

## Download the bomb binary

CTU Students: Docker container. ▾

MOOC Students: SCL Reversing-Lab ▾

The binary is also in the class folder in Docker. Let's execute it and try some ideas:

- `cd ~/reversing-class/second_example`
- `chmod +x bomb_64`
- `./bomb_64`
  - Input some data and press Enter
- Run the command "file":
  - `file bomb_64`

bomb\_64: **ELF 64-bit** LSB shared object, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=7dd166a66acce52fc6103bbf61a0c32b7e667841, for GNU/Linux 3.2.0, with debug\_info, not stripped

- Run the command strings:

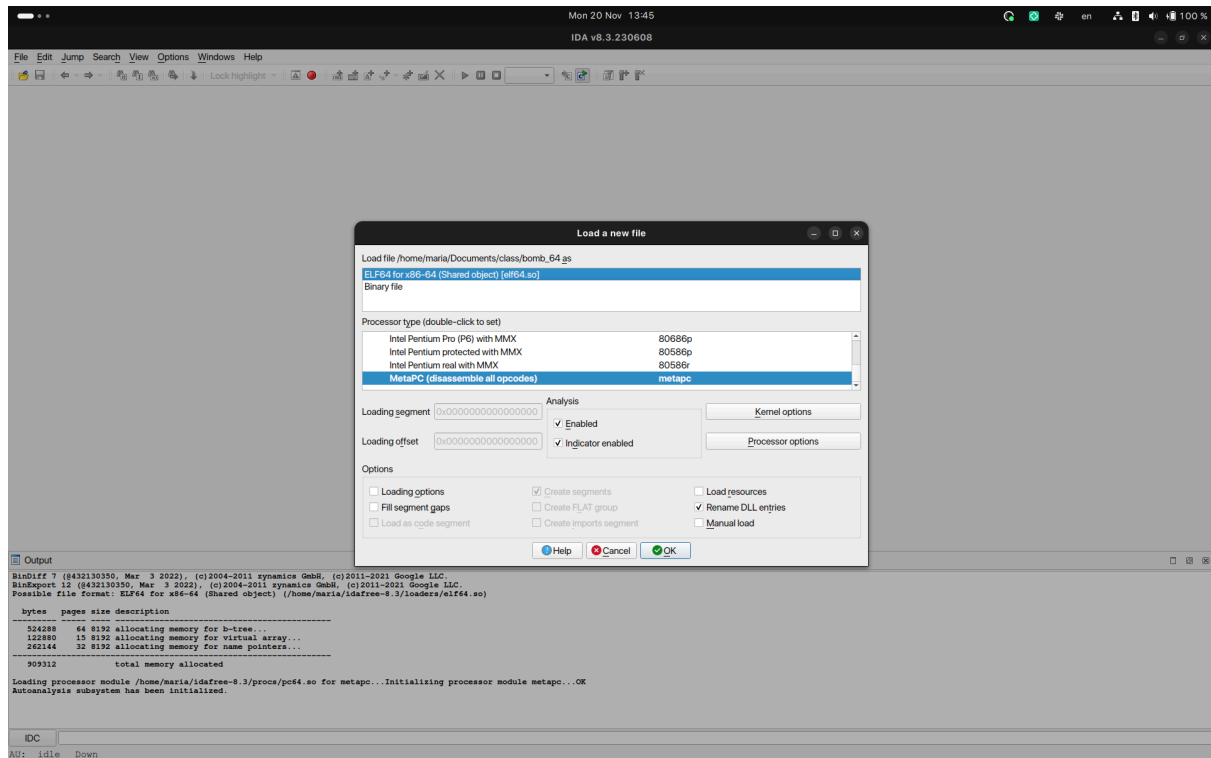
<sup>14</sup> <http://csapp.cs.cmu.edu/3e/labs.html>

- `strings bomb_64 | more`

Host computer. Not in container. ▾

Download the file **to your laptop** from [here](#).

Open IDAFree and **load** the binary file. Press “**OK**” on the first screen and “**Yes**” on the next DWARF window.



You will be greeted with the disassembly view of the **main** function as a Graph.

1. Press “**space**” to see the disassembled function listing. Pressing “**space**” again will return to the Graph view.
2. If you double-click on a variable or a function name, it will take you to its definition. Pressing “**Esc**” takes you back to the previous function.
3. There are multiple views:
  - a. “**Imports**” shows the imported libraries
  - b. “**Exports**” shows the exported functions.
4. Press **Shift-F12** to open the “Strings” view
5. Press **Shift-F7** to open the “Segments” view.
6. You can find all available views in **View -> Open subviews**



7. Go back to the 'IDA View-A' TAB.
8. Press **F5** while in a disassembly window (IDA View-A) to open the decompilation view.

You can discover more shortcuts in this [cheatsheet](#)<sup>15</sup>.

**Note:** If F5 does not work due to networking issues (cloud decompilation), close IDA, download the database from [here](#), and save it in the same folder as the binary. Then, open the database file either by double-clicking it (Windows) or by opening it through IDA.

## Main function

The list of disassembled functions is on the left panel. If you scroll up and down, you can see

- libc functions
- Bomb related functions (*phase\_1()*, *main()*, *phase\_defused()*, etc)

Let's first look at the `main` function. The first part checks the input arguments:

- If no extra argument is provided, the program will read from standard input.
- If there is more than 1 argument, it will print the usage.
- If there is one argument, it expects it to be a file and it will attempt to read it.

```
if ( argc == 1 )
{
    infile = (FILE *)stdin;
}
else
{
    if ( argc != 2 )
    {
        __printf_chk(1LL, "Usage: %s [<input_file>]\n", *argv);
        exit(8);
    }
    infile = fopen(argv[1], "r");
    if ( !infile )
    {
        __printf_chk(1LL, "%s: Error: Couldn't open %s\n", *argv, argv[1]);
        exit(8);
    }
}
```

<sup>15</sup>

<https://elhacker.info/Cursos/%5BTutsNode.com%5D%20-%20Reverse%20Engineering%20IDA%20For%20Beginners/04%20Basic%20File%20Analysis%20and%20IDA%20Usage/008%20IDA%20Cheatsheet-v3.pdf>

💡 What is the high-level structure of the program? 💡

In the second part, there is an initialization function and some messages that are printed on screen using `puts`. Then, the program reads a line (using `read_line`) and proceeds through the six phases.

After each phase, there is a `phase_defused()` function.

```
initialize_bomb();
puts("Welcome to my fiendish little bomb. You have 6 phases with");
puts("which to blow yourself up. Have a nice day!");
line = (const char *)read_line();
phase_1((__int64)line);
phase_defused(line);
puts("Phase 1 defused. How about the next one?");
v4 = (const char *)read_line();
phase_2((__int64)v4);
phase_defused(v4);
puts("That's number 2. Keep going!");
v5 = (const char *)read_line();
phase_3((__int64)v5);
phase_defused(v5);
puts("Halfway there!");
v6 = (const char *)read_line();
phase_4((__int64)v6);
phase_defused(v6);
puts("So you got that one. Try this one.");
v7 = (const char *)read_line();
phase_5((__int64)v7);
phase_defused(v7);
puts("Good work! On to the next...");
v8 = (const char *)read_line();
phase_6((__int64)v8);
phase_defused(v8);
return 0;
```

## initialize\_bomb()

Double click on the function name. Press F5 to get the C code. It defines a signal handler for `signal16 2 (SIGINT17)` that defines how to behave when “Ctrl-C” is pressed:

```
_sighandler_t initialize_bomb()
{
    return signal(2, sig_handler);
}
```

This is the `sig_handler`. Double-click on this function name, `sig_handler`.

<sup>16</sup> [signal\(2\)](#)

<sup>17</sup> [signal\(7\)](#)

```

void __noreturn sig_handler()
{
    puts("So you think you can stop the bomb with ctrl-c, do you?");
    sleep(3u);
    __printf_chk(1LL, "Well...");
    fflush(_bss_start);
    sleep(1u);
    puts("OK. :-)");
    exit(16);
}

```

CTU Students: Docker container. ▾

MOOC Students: SCL Reversing-Lab ▾

We can test this by running the bomb and pressing Ctrl-C instead of giving the input.

```
./bomb_64
```

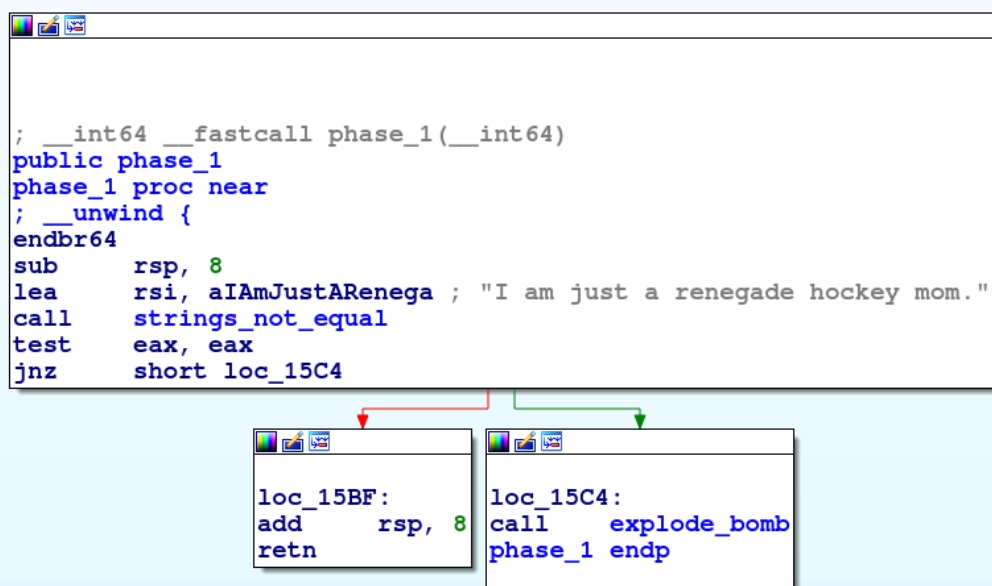
```
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
```

```
^CSo you think you can stop the bomb with ctrl-c, do you?
```

```
Well...OK. :-)
```

## Phase 1

Go to the 'IDA View-A' TAB and then double-click on the **phase\_1** function call:



We can simply look at the decompiled version or even figure out the solution from the graph, but that wouldn't be fun. Let's proceed by examining the disassembly only. Step by step...



*Everyone's face at this moment*

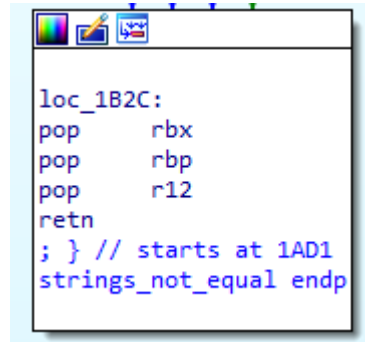
We notice a call to the `strings_not_equal()` function. Let's follow it by double-clicking on it.

We see something like this:

```
; __int64 __fastcall strings_not_equal(char *, _BYTE *)
public strings_not_equal
strings_not_equal proc near
; __unwind {
endbr64
push    r12
push    rbp
push    rbx
mov     rbx, rdi
mov     rbp, rsi
call    string_length
mov     r12d, eax
mov     rdi, rbp
call    string_length
mov     edx, eax
mov     eax, 1
cmp     r12d, edx
jnz     short loc_1B2C
```

What are the first three **push** commands for?

This is what we call the **prologue** of a function. At the **epilogue** of a function, we restore these values **in reverse** order. We do this with the **pop** command.



```

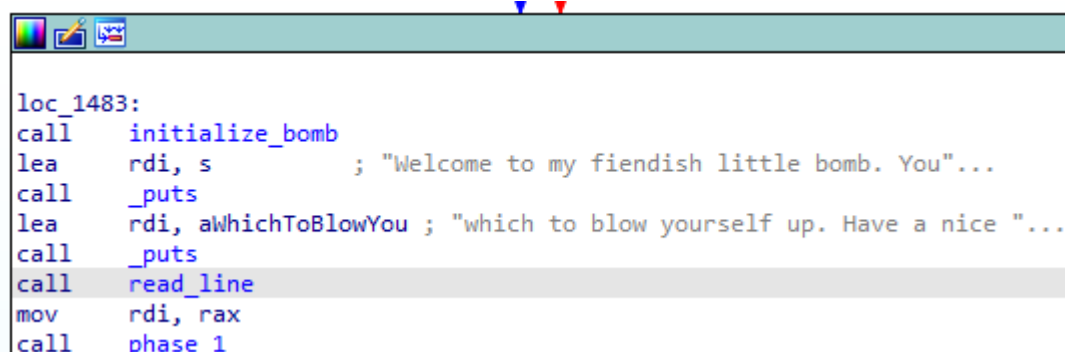
loc_1B2C:
pop     rbx
pop     rbp
pop     r12
retn
; } // starts at 1AD1
strings_not_equal endp
    
```

Now that we know how to recognize the prologue and epilogue of a function, we can examine what happens in between.

We notice that we are moving the value of the **rdi** register to **rbx**, and the **rsi** to **rbp**.

What is written in **rdi** and **rsi**? Perhaps we should backtrack a bit...

- Let's go back to the `phase_1()` function by pressing **Esc** on your keyboard.
- Notice that before the function call, we put something in the **rsi** register.
  - It is a string: *"I am just a renegade hockey mom."*
- But what is in **rdi**?
- Let's go even further back, to the main function
  - Press **Esc** again



```

loc_1483:
call    initialize_bomb
lea     rdi, s          ; "Welcome to my fiendish little bomb. You"...
call    _puts
lea     rdi, aWhichToBlowYou ; "which to blow yourself up. Have a nice "...
call    _puts
call    read_line
mov     rdi, rax
call    phase_1
    
```

You can notice that we move **rax** to **rdi** before we call the function. So what is in **rax**?

Notice the `read_line` function before `mov`. As we reminded ourselves at the beginning of the class, **rax** register is the *return value register*. What could be the return value of

the *read\_line()* function? Well, most likely our input. So now we know what is in **rdi** and **rsi**.

We can continue with Phase 1.

We notice that the next call is to the *string\_length()* function. Furthermore, this function is called twice. We will not analyze these functions in such great detail (yay), but in a nutshell, they return the length of the string, as the function name suggests.

Once we know the length of both strings, we compare them. The **jnz** command tells us that if the lengths are not equal, we jump to the end of the function and return. Notice that before **cmp**, the function writes **1** to **eax**. If the jump happens, 1 is returned from the function.

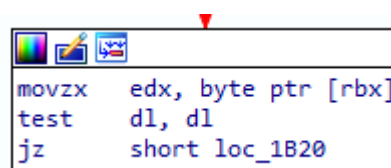
In the *phase\_1* function, there is a **test eax, eax**. If the result is not zero, the bomb explodes.

```

public phase_1
phase_1 proc near
; __unwind {
endbr64
sub     rsp, 8
lea     rsi, aIAmJustARenega ; "I am just a reneg
call    strings_not_equal
test    eax, eax
jnz     short loc_15C4

```

We double-click on *strings\_not\_equal()*. We see, therefore, that the strings must be equal in length.

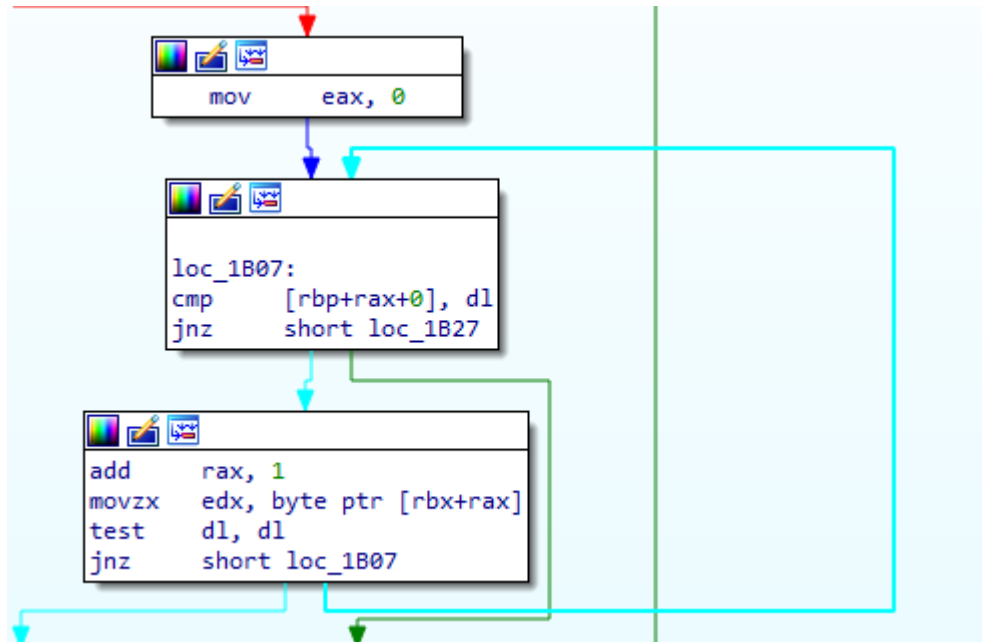


```

movzx   edx, byte ptr [rbx]
test    dl, dl
jz      short loc_1B20

```

Then it loads the first byte of **rbx** to **edx**. We then check the lower 8 bits of **edx** (**dl**) if the first byte is null. If it is, zero is returned. If not, we continue comparing strings.



Notice that there is a loop here. What does it do?

1. First, it compares the byte in the **edx** register with the byte at **rbp + rax**. If they are not equal, the function jumps and returns 1.
2. If they are equal, **rax** is increased by 1. So we can conclude that **rax**, in this case, holds the value of the counter, and we increment it in each step. This is some kind of while loop or for loop.
3. If both strings reach the end, and the last comparison of characters, both are **\0**, we return 0 (zero) from the function!

This function actually compares whether two strings are equal. Who would have guessed? 😊

CTU Students: Docker container. ▾

MOOC Students: SCL Reversing-Lab ▾

We know that `strings_not_equal()` checks if the input is equal to a constant string stored in memory. Let's try the string:

```
root@bsylabs:~/reversing-class/third_ex$ ./bomb_64
```

```
Welcome to my fiendish little bomb. You have 6 phases with
```



which to blow yourself up. Have a nice day!

I am just a renegade hockey mom.

Phase 1 is defused. How about the next one?

Now that we have the correct string, we can use the input file to avoid typing.

- `echo "I am just a renegade hockey mom." > bomb64_input.txt`
- `./bomb_64 bomb64_input.txt`

Solution:



## Phase 2

Go back to the `main()` function in IDA by pressing **Esc** or by selecting the function from the left side window. Double-click on the `phase_2()` function and then **F5** to view the decompiled version:

```

unsigned __int64 __fastcall phase_2(__int64 a1)
{
    char *v1; // rbx
    _DWORD v3[5]; // [rsp+0h] [rbp-38h] BYREF
    char v4; // [rsp+14h] [rbp-24h] BYREF
    unsigned __int64 v5; // [rsp+18h] [rbp-20h]

    v5 = __readfsqword(0x28u);
    read_six_numbers(a1, (__int64)v3);
    if ( v3[0] != 1 )
        explode_bomb(a1);
    v1 = (char *)v3;
    do
    {
        if ( *((_DWORD *)v1 + 1) != 2 * *((_DWORD *)v1) )
            explode_bomb(a1);
        v1 += 4;
    }
    while ( v1 != &v4 );
    return __readfsqword(0x28u) ^ v5;
}

```

Phase\_2 analysis:

- It calls the function *read\_six\_numbers()*
- *read\_six\_numbers()* uses *sscanf* to read six integers from the input line (a1) and stores them in the a2 array:

```

__int64 __fastcall read_six_numbers(__int64 a1, __int64 a2)
{
    __int64 result; // rax

    result = __isoc99_sscanf(a1, "%d %d %d %d %d %d", a2, a2 + 4, a2 + 8, a2 + 12, a2 + 16, a2 + 20);
    if ( (int)result <= 5 )
        explode_bomb(a1);
    return result;
}

```

- After that, the *phase\_2* function checks if the first number in the array is equal to 1.
- Then the remaining numbers are expected to be the previous number multiplied by 2 (while loop)

CTU Students: Docker container. ▾

MOOC Students: SCL Reversing-Lab ▾

- `echo " " >> bomb64_input.txt`

## The power of AI

- Let's see what AI thinks about this. Create a new file:
  - `vim phase2.c`
- Copy the decompiled code from IDA into the file.
- Ask bsy-clippy

CTU Students: Docker container. ▾

- `cat phase2.c | bsy-clippy --profile cesnet -r 5 -c -u "What does this C code do?"`

MOOC Students: SCL Reversing-Lab ▾

- `cat phase2.c | bsy-clippy -r 5 -c -u "What does this C code do?"`

Solution:



## Phase 3

The assembly graph has a very interesting structure. It is a `switch/case` construct.

The decompiled listing shows us that `sscanf()` is used to read the next input and that two integers are expected. The switch checks the first number (`v11`).

In the end, there is also a check that requires `v11` to be less than 5; otherwise, the bomb will explode.

```

if ( (int) __isoc99_sscanf(a1, "%d %d", &v10, &v11) <= 1 )
    explode_bomb(a1);
switch ( v10 )
{
    case 0:
        v1 = 628;
        goto LABEL_5;
    case 1:
        v1 = 0;
LABEL_5:
        v2 = v1 - 588;
        goto LABEL_6;
    case 2:
        v2 = 0;
LABEL_6:
        v3 = v2 + 688;
        goto LABEL_7;
    case 3:
        v3 = 0;
LABEL_7:
        v4 = v3 - 126;
        goto LABEL_8;
    case 4:
        v4 = 0;
LABEL_8:
        v5 = v4 + 126;
        goto LABEL_9;
    case 5:
        v5 = 0;
LABEL_9:
        v6 = v5 - 126;
        goto LABEL_10;
    case 6:
        v6 = 0;
LABEL_10:
        v7 = v6 + 126;
        break;
    case 7:
        v7 = 0;
        break;
    default:
        explode_bomb(a1);
}
v8 = v7 - 126;
if ( v10 > 5 || v11 != v8 )
    explode_bomb(a1);
return __readfsqword(0x28u) ^ v12;
}

```

## LESSON 9 / REVERSE ENGINEERING

We can work with the value 4 for the first number to minimize the number of jumps and calculate the required value of the second number.

We can also use GDB to easily find the value of `v12` without any calculations. Let's put the values 4 and 100 in the `bomb64_input.txt` and then run GDB:

- `echo "4 100" >> bomb64_input.txt`
- `gdb ./bomb_64`
- `break phase_3`
- `run < bomb64_input.txt`
- `disassemble`

Set another breakpoint before the first check that triggers the bomb's explosion.

- `b *0x<address_of_instruction>`
- `continue`
- `ni`
  - `ni`: Run until next instruction. Steps over calls.

Just before the second `cmp` instruction, we can check the contents of the `rax` register that contains the expected value. We can set the value in the stack to pass the check:

- `x/x $rsp+4`
- `set *<address_of_rsp+4>=0`
- `x/x $rsp+4`
- `c`

Solution:



~~~~  **Second Break!**  ~~~~ (16:40, 10m)

Phase 4 (16:50)

Looking at the `sscanf()` function, the main function of `phase_4()` expects two integers. The first one (`v3`) needs to be less than or equal to 14 (0xE).

Then it calls another function, `func4()`, with `v3` as a parameter and expects the result to be 10. Similarly, the second integer (`v4`) is also checked, and the expected result is 10.

```
if ( (unsigned int)__isoc99_sscanf(a1, "%d %d", &v2, &v3) != 2 || v2 > 0xE )
    explode_bomb();
if ( (unsigned int)func4(v2, 0, 14) != 10 || v3 != 10 )
    explode_bomb();
```

This is what the decompiled listing for `func4()` looks like:

```
__int64 __fastcall func4(__int64 a1, __int64 a2, __int64 a3)
{
    int v3; // ebx

    v3 = a2 + ((int)a3 - (int)a2) / 2;
    if ( v3 > (int)a1 )
    {
        v3 += func4(a1, a2, (unsigned int)(v3 - 1));
    }
    else if ( v3 < (int)a1 )
    {
        v3 += func4(a1, (unsigned int)(v3 + 1), a3);
    }
    return (unsigned int)v3;
}
```

CTU Students: Docker container. ▾

MOOC Students: SCL Reversing-Lab ▾

It is a recursive function, and it can be tricky to understand. But we can always rewrite it in Python and brute-force, which input value returns the value 10:

- `python3 phase4.py`

Or we can ask our beloved Clippy to help us:

LESSON 9 / REVERSE ENGINEERING

First copy the content of *phase_4()* and *func4()* to a file:

- `vim phase4.c`

The run something like the following:

CTU Students: Docker container. ▾

- `cat phase4.c | bsy-clippy --profile cesnet -r 5 -c -u "What does this C code do?"`

MOOC Students: SCL Reversing-Lab ▾

- `cat phase4.c | bsy-clippy -r 5 -c -u "What does this C code do?"`

Solution:



Phase 5

```

    if ( (int)__isoc99_sscanf(a1, "%d %d", &v5, &v6) <= 1 )
        ((void (__noreturn *) (void))explode_bomb)();
    v1 = v5 & 0xF;
    v5 = v1;
    if ( v1 == 15 )
        goto LABEL_7;
    v2 = 0;
    v3 = 0;
    do
    {
        ++v3;
        v1 = array_3471[v1];
        v2 += v1;
    }
    while ( v1 != 15 );
    v5 = 15;
    if ( v3 != 15 || v6 != v2 )
LABEL_7:
    explode_bomb(a1);

```

The level expects two integers (v5 and v6). The first one (v5) needs to be less than 15 (0xF). The main loop replaces the first number with the contents of `array_3471`, which is an array of 16 numbers from 0 to 15. None of the numbers is repeated.

Double-click on the array to see its contents:

```

.rodata:000000000000031C0 ; _DWORD array_3471[16]
.rodata:000000000000031C0 array_3471 dd 0Ah, 2, 0Eh, 7, 8, 0Ch, 0Fh, 0Bh, 0, 4, 1, 0Dh, 3, 9
.rodata:000000000000031C0 ; DATA XREF: phase_5+49↑
.rodata:000000000000031F8 dd 6, 5

```

The loop uses v1 as an index to the array, cycling through all the values and stopping when the value of v1 becomes 0xF. The check expects the loop to be run 15 times and for the sum of the numbers to equal the second integer input.

💡 Try to work out the solution. Use any way that you like, pen and paper, code, GDB, reasoning, LLMs 💡

Notice that:

- a) **The loop has to be executed 15 times**
- b) **The last value has to be 0xF.**
- c) **We need to find where to start to end up at 0xF**
- d) **The code goes through all the elements of the array. If it didn't and there was a repetition, it could not have reached 0xF in 15 steps.**
- e) If we allow it to run more than 15 steps, it will repeat the cycle.
- f) Doesn't that mean that the start number should be where 0xF points to (5)?

The second number is the sum of all the numbers we go through in the loop. Is it the sum of the whole array (120)? We can use GDB to find out.

CTU Students: Docker container. ▾

MOOC Students: SCL Reversing-Lab ▾

We set v5 to 5, and we will set v6 to 120 to start the analysis:

- `echo "5 120" >> bomb64_input.txt`
- `gdb ./bomb_64`
- `break phase_5`
- `run < bomb64_input.txt`
- `ni`
- `disassemble`

Find the address before the first `call explode_bomb` instruction and add a breakpoint in the `cmp` instruction before it:

- `b *<address_before_explode_bomb>`
- `continue`
- `ni`
- `x/x $rsp+4`

Check the value of the `ecx` register to see what is expected:

- `p $ecx`
- `set *<address_of_rsp+4>=0x73`

💡 Why is this the correct value? 💡

Solution:

Phase 6

This one is the most complicated.

After some renaming of variables, the first part of the listing looks like this:

```
copy_input_array = input_array;
read_six_numbers(a1, input_array);
for ( i = 1; ; ++i )
{
    if ( (unsigned int)(*copy_input_array - 1) > 5 )
        explode_bomb();
    if ( (int)i > 5 )
        break;
    idx = i;
    do
    {
        if ( *copy_input_array == input_array[idx] )
            explode_bomb();
        ++idx;
    }
    while ( (int)idx <= 5 );
    ++copy_input_array;
}
```

The input is expected to be 6 integers. The first loop checks that all numbers are less than or equal to 6 and that they are all distinct from each other.

We are now examining the permutations of the numbers 1 through 6.

The key to understanding the rest of the code is to first identify the **node1** struct that we see in the listing.

- Double-click on **node1** in IDA
- The variable is stored in the **.data** section, along with five other nodes.
- It is a struct with the following format:

```

struct node {
    int value;

    int id;

    struct node *next;
};

```

This is how the node structs look in memory, using GDB:

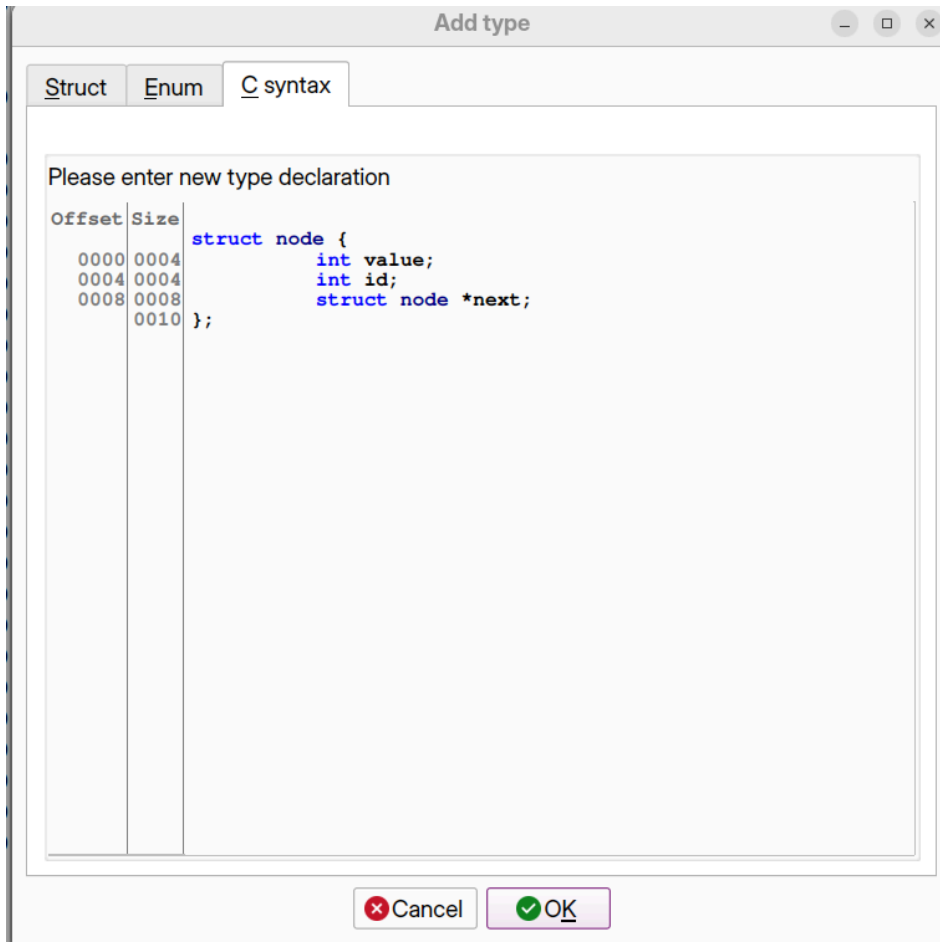
```

gef> x/20x 0x0000558821125200
0x558821125200 <node1>: 0x00000212      0x00000001      0x21125210      0x00005588
0x558821125210 <node2>: 0x000001c2      0x00000002      0x21125220      0x00005588
0x558821125220 <node3>: 0x00000215      0x00000003      0x21125230      0x00005588
0x558821125230 <node4>: 0x00000393      0x00000004      0x21125240      0x00005588
0x558821125240 <node5>: 0x000003a7      0x00000005      0x21125110      0x00005588
gef> x/4x 0x0000558821125110
0x558821125110 <node6>: 0x00000200      0x00000006      0x00000000      0x00000000

```

IDA allows us to create custom structs and use them to guide the disassembly and decompilation. We will need to go to the **Views -> Open subviews -> Local Types tab (Shift+F1) and press the Ins key to add a new structure** (or right-click in the variable list and then -> Add Type)

Copy the code that defines the structure from above to the “**C syntax**” tab and press ok.



- In the code listing on line 50, right-click on the v7 variable, select “**Convert to struct***”, and then select node.
- Do the same with the v8 variable.

The do...while loop at the end is very interesting. It traverses the linked list and checks if the value of the next node is greater than that of the current node. And it explodes if it is:

```
do
{
    if ( v8->value < v8->next->value )
        explode_bomb();
    v8 = v8->next;
    --v14;
}
while ( v14 );
```

💡 What if we just need to order the nodes in descending order of their values? 💡

LESSON 9 / REVERSE ENGINEERING

Solution:



Assignment 7 (4 Points)

1. The assignment has two parts. 2 points each.
2. The first part will test your prowess with the network.
3. The second part will test your skills in reversing binaries.

DEADLINE: 07/01/2026 23:59:59 CET



Class Feedback

By giving us feedback after each class, we can make the next class even better!

<https://bit.ly/BSY-Feedback>

